QAgent: 30-minute demo pack (Techathon 2026, Team Cynacs)

For the presenters. Sources: the two team slides, `demo-ecoforecast/DEMO.md`, `CONTEXT.md`, the saved plan "EcoForecast PRD plan (demo)" and the judging-criteria page. Demo Day is 13-14 October 2026.

The brief from the organisers: 1-2 intro slides, then straight into the demo, time left for Q&A, a backup ready, and a demo that shows **business value, a working POC and reusability**.

---

0. The story in one breath

> Software that produces numbers people make decisions on is rarely tested the way the business uses it. QAgent lets anyone describe, in plain English, what a product must do. An AI then drives a real browser, checks the result, and reports each bug with proof. We proved it on our own forecasting product, ForecastIQ (EcoForecast), where it found real bugs a clean-data test would never find.

Three ideas to repeat (leadership remembers three things): **plain English in, proof out, replay for free.**

---

1. Run of show (30 min)

Min	Part	Who/what	Goal
0:00-0:03	Slides 1-2 (problem, solution)	Presenter A	Frame the problem, name the three outcomes: speed, proof, reuse
0:03-0:05	**A. The app under test**	Presenter B, ForecastIQ	Show it working first, so the bugs mean something
0:05-0:07	**B. Set up an app in minutes**	QAgent Settings	Zero-code onboarding, safe sign-in
0:07-0:11	**C. Plan from the product document**	QAgent, "Create from product document"	Requirements and goal-style journeys in about 30 s
0:11-0:14	**D. Partial testing and pre-sales prototypes**	QAgent, short notes to a one-journey plan, run	Test one flow, not the whole product
0:14-0:18	**E. A real run and what it found**	Live view, then the saved full run	Proof behind each bug
0:18-0:20	**F. Bugs over time and free replay**	Bugs page, "Repeat exactly"	Fix, re-run, confirm
0:20-0:21	**G. Requirements coverage**	Requirements page	"Is every requirement tested?"
0:21-0:22	**H. Value, reuse, next steps**	Slide 2 or the dashboard	Close on the ask
0:22-0:30	**Q&A**	Everyone	Section 5 has prepared answers

Time is tight. If you run long, cut G and shorten B (the Settings page is a screenshot-level point).

Run in the background: start the saved full plan in Smart mode at about minute 5 and let it work while you talk. A full run takes about 10 minutes (65 steps, about $1.50). Do not start it in parallel with the quick check in D; the backend is a small 2-core machine and runs go one journey at a time. Order: quick check in D first, then the big run, then show the finished run from earlier for E and the new one if it is done.

---

2. Talking points by segment

Format: **Say** (the words), **Do** (the clicks), **Show/Proof** (what the audience should see), **Fallback**.

A. The app under test (2 min)

- **Say:** "This is ForecastIQ, our agentic short-term forecasting tool. A commercial analyst uploads brand data, picks a 12-month horizon, validates it, and picks a method. Here, Ocrevus, six territories. The forecast is 6,786 now, 6,889 at the end, a 1.5% rise. Everything looks fine, and that is exactly the problem: nobody checks the cases that are not fine."
- **Do:** sign in as admin, 12 months, upload `ocrevus_trx_data.csv`, "6/6 checks pass", Explore, Method (ETS), Forecast.
- **Proof:** the numbers 6,786 and 6,889 (they come back later as an expected outcome in the plan).
- **Fallback:** the screen recording in `demo-video/footage/eco_app.mp4`.

B. Set up an app in minutes (2 min)

- **Say:** "Setup is three things. The web address. The sign-in, saved once and encrypted; the AI only ever sees the field names, never the password, and it is masked in every report. And the test files: the clean file and deliberately broken ones, an empty file, a header-only file, a short history."
- **Do:** open EcoForecast > Settings: show allowed hosts, sign-in details, test files.
- **Point to make:** no code, no selectors, no test scripts.
- **Fallback:** screenshot of Settings.

C. A test plan from the product document (4 min)

- **Say:** "Teams already have product requirement documents. QAgent reads one and, in about thirty seconds, drafts the test plan: the requirements, and journeys written as goals, like 'upload the clean file, choose ETS, reach the forecast', with what must be true afterwards. Nothing says 'click the third button'. So when the page changes, the plan does not break. That is where scripted tests fall over."
- **Do:** Test plans > Add a test plan > *Create from product document* > paste `demo-ecoforecast/PRD.md`. While it works, open the saved plan "EcoForecast PRD plan (demo)" (version 5) to show the result: requirement cards, journeys, "Should be true afterwards".
- **Proof:** an expectation like "national baseline 6,786, projected horizon about 6,889, net growth about 1.5%". **Say:** "The numbers are checked by code, not by the AI's opinion."
- **Fallback:** if generation is slow, say "this is the reviewed version of what it generates" and use the saved plan.

D. Partial testing and pre-sales prototypes (3 min)

This is the segment that answers "do I have to test everything?". The answer is no. - **Say:** "You do not need a full document. Suppose a sales team has a prototype for a client demo tomorrow. Write three lines about the one story you are going to show. QAgent turns it into a small plan for exactly that flow, and tests only that." - **Do:** Test plans > Add a test plan > paste three lines, for example: *"Sign in. Choose a 12-month horizon. Upload ocrevus_trx_data.csv. Confirm the validation shows 6 of 6 checks passed." > Rewrite my notes as a test plan* > Save > Run now > Smart mode (about a minute and a half). - **Proof:** the live view fills with steps, each showing what the AI chose and its confidence. - **Say (value):** "Before a pre-sales prototype goes in front of a client, we point QAgent at it and check the one story we are about to show. Minutes, not a QA cycle." - **Fallback:** an older small plan exists ("Genentech File Upload and 18-Month Forecasting", 5 journeys; "CSV File Upload and Analysis with Canada Credentials"). Open its finished run.

E. A real run and what it found (4 min)

- **Say (how it works):** "QAgent opens a real Chromium, reads the page, and a fast AI, Jev, picks which element does each step in about 300 milliseconds. When it is unsure, a stronger AI, Claude, double-checks. That keeps it quick and cheap. In the last full run: 64 fast decisions, none needed an escalation."
- **Do:** open the finished full run (`02475b32`): 10 journeys, 65 steps, 12 problems, $1.50. Journeys tab, then Problems tab.
- **The three headline bugs (all real, from DEMO.md):** 1. **A blank file.** The server answers 422 (rejected), but the page shows "parsed" with 144 invented rows. *Say:* "The system said no and the screen said yes. A clean-data test never sees this." 2. **Export Forecast (.csv) does nothing.** No download, no server request. 3. **A flat 0% custom curve is still ranked "Best Fit"** and the Auto-select option picks it.
- **Do:** click *See the proof* on the 422 problem: the annotated screenshot, the 422 call, "what the AI chose before this".
- **Say:** "Every bug carries its own proof: screenshot, server call, steps to reproduce. Developers do not have to take our word for it."
- **Be honest about noise:** "Slow-step warnings like 'step exceeded 2000 ms' are real but low-severity; the benchmark and the slide deck genuinely take several seconds."
- **Fallback:** the video, or screenshots of the Problems tab and proof panel.

F. Bugs over time and free replay (2 min)

- **Say:** "The same problem across runs is one bug. It shows as open, came back, or fixed once a later run no longer finds it. After a fix, *Repeat exactly* replays the recorded decisions: no model calls, so it costs nothing, and it is the same test every time."
- **Do:** Bugs page (16 bugs for this app; filter Open / Came back / Fixed), then a run > *Repeat exactly*.
- **Say (value):** "That is the loop: find, fix, replay, confirm, at no extra AI cost."

G. Requirements coverage (1 min)

- **Do:** Requirements page, "Is every requirement tested?".
- **Say:** "Leadership can ask a simple question: which requirements are tested, and which have a bug against them."

H. Value, reuse, next steps (1-2 min)

- **Numbers to quote** (all from the product, state them with their source):
- One full plan run: 10 journeys, 65 steps, about 10 minutes, about $1.50 (about $0.28 per run on average across our 46 runs in 30 days, which include development runs).
- Our own testing so far: 178 problems found across 3 apps.
- "About 60% less manual regression effort per release" is an **estimate** on the slide. **Prepare the basis before Demo Day**: time one person doing the same 10 journeys by hand once, and quote that against the 10-minute run.
- **Reuse:** the engine is app-agnostic. A new app is a web address, a sign-in and a plan. The plan can come from a PRD in 30 seconds.
- **Next steps (ask):** 1. Pilot on 1-2 Chryselys client web apps. 2. Hook runs into the release pipeline (a CI workflow is already written, `deploy/ci.github-actions.yml`). 3. Pre-sales prototype checks as a standard step before client demos. 4. A shared library of plans and test files per product family. 5. Per-user accounts and an audit trail (today: one shared account).

---

3. Before you go on: checklist (do this 20 minutes before)

- [ ] **Backend awake.** The QAgent API runs on a GitHub Codespace that **stops when idle** (it was stopped when we last checked). Start it and check `https://qagent-web-mu.vercel.app/api/health`: `postgres`, `storage`, fast AI and strong AI must all be `true`. Procedure: `deploy/redeploy.sh be` (needs the GitHub CLI signed in).
- [ ] Open https://ecosystem-forecast.vercel.app once and sign in (admin); wake its backend.
- [ ] QAgent: sign in, open EcoForecast, confirm Settings shows both allowed hosts (`ecosystem-forecast.vercel.app, *.sslip.io`), the saved sign-in and the test files.
- [ ] Dismiss the page tours ("Skip") so they do not pop up mid-demo.
- [ ] Do one rehearsal of D and the first journeys of the big run (about 4 minutes).
- [ ] Browser zoom 110-125%, one window, notifications off, second tab pre-opened on the finished run `02475b32` and on the Bugs page.
- [ ] **Backup open and ready:** the screen-recording video (`demo-video/out/`), screenshots of each step, the finished run URLs.
- [ ] Decide who speaks and who clicks.

4. If something fails live

Symptom	Do
QAgent looks empty / "Something went wrong"	The backend is asleep. Switch to the video, say "this is the recording of the same flow", and restart the Codespace in the background
A run is slow to start	Talk through the plan; the first step can be slow when the app's backend is cold
A journey fails oddly	Open the run's activity log: a "blocked host" warning means an allowed host is missing
Result differs from the script	Say so. AI runs vary slightly (earlier runs of the same plan found between 11 and 13 problems); the saved run is the reference

---

5. Likely questions from leadership, with answers

Business and value 1. *What problem does this solve that we do not already cover?* Manual regression is slow and scripted UI tests break when pages change. QAgent tests from plain-English goals, so pages can change without rewriting tests, and every bug has proof. 2. *What does it cost?* A full 10-journey run is about $1.50 and about 10 minutes. The average across our 46 runs is $0.28. Replays cost nothing. Practice mode (no AI) is free. 3. *Does it replace QA engineers?* No. It takes the repetitive regression and the first pass off them, so they spend time on judgement and exploratory testing. 4. *Where does the "60% less effort" come from?* It is an estimate. (Have the measured basis ready; see H.) 5. *Who would use it?* Developers, QA and delivery teams on Chryselys and client web apps, and sales engineers checking a prototype before a client demo. 6. *Can we test just one flow?* Yes. A plan can be three lines about one journey (segment D). That is how a pre-sales prototype gets checked in minutes. 7. *What is the next step if we like it?* A pilot on 1-2 client apps, then a release-pipeline hook.

Trust and accuracy 8. *How do we know the AI is not inventing bugs?* Every bug carries proof: screenshot, the actual server calls, the steps. Numbers are checked by code against formulas in the plan, not by the AI's opinion. A stronger AI double-checks when the fast one is unsure. 9. *Will it give the same answer every time?* The AI's choices can vary slightly between fresh runs. *Repeat exactly* replays the recorded decisions for a deterministic re-test, which is how you confirm a fix. 10. *What about false alarms?* Some low-severity speed warnings appear every run (a benchmark that genuinely takes seconds). Bugs have a severity so they can be filtered. Tuning is part of a pilot. 11. *Which of the bugs found were real?* The empty-file 422, the dead export button and the best-fit ranking are the ones documented in `DEMO.md` as real problems on the deployed app. **Before Demo Day, reproduce each one by hand once** so you can say you did.

Security and compliance 12. *What happens to the sign-in details?* Stored encrypted per app, the AI sees only field names, and they are masked in every recorded place. 13. *Where does our data go?* The page content goes to the AI services (a fast model and Claude on AWS Bedrock). Use test or approved data, which is also in our own guidelines. Test files are stored per app and have a retention policy. 14. *Is it validated for regulated (GxP) use?* No. It is a testing accelerator, not a validated system. Today it uses one shared team account with no per-user audit trail; that is a known gap and a next step. 15. *SSO or multi-factor login?* We have only proven username-and-password sign-in. We would scope SSO/MFA in a pilot rather than promise it.

Technical 16. *How does it know what to click?* It lists the real elements on the page, a fast AI picks one for each step, and a stronger AI checks when unsure. Goal-style journeys have an AI "director" choose each next step from the live page. 17. *What if the UI changes?* Goal journeys describe intent, not selectors, so they usually survive. Steps that need exactness can still be written out. 18. *What can it test?* Web apps in a real browser. Not mobile apps or desktop software today. 19. *Does it scale?* Each app is independent. Today one API worker runs the live events, so concurrent heavy use needs a small change; that is on the list. 20. *Which models does it depend on?* A fast model (Jev) and Claude. The decision code is kept separate from the browser code in the codebase, so changing model is a contained change. We have not tested other vendors. 21. *Can it run in our CI?* A workflow is written and ready; adding it is a next step. 22. *What input formats?* Markdown or text product documents (no PDF or Word yet), or short notes typed or dictated.

Competition and novelty 23. *How is this different from Selenium, Cypress or Playwright?* Those need someone to write and maintain scripts. QAgent needs a plain-English plan, adapts to page changes, and produces a proof-backed report. 24. *How is it different from record-and-playback tools?* Recordings break on every layout change. Ours are goal-driven, with a replay only for exact re-tests. 25. *What was the*

hardest part? Making AI actions reliable: the AI chooses among stamped elements and the executor acts on exactly one, never "the first match".

---

6. Mapping to the judging criteria

Know these cold. For each criterion, say it once in the demo and have the evidence ready.

Criterion (weight)	What to say	Evidence on screen	Risk to pre-empt
Solution-Oriented (25%): real problem, practical solution, working POC, scalability	"Regression testing is slow, scripted tests break, bugs lack proof. This is a working, deployed app that found real bugs in our own product."	Live web app (qagent-web-mu.vercel.app), a real run, 12 problems with proof, deployed backend, test suite (114 backend, 51 unit, 80 end-to-end tests)	Backend asleep (checklist). Scalability: say honestly "one API worker today; a known, small change"
Chryselys Business Value and Reusability (25%)	"A reusable QA accelerator: any web app is a URL, a sign-in and a plan. Plans can come from a PRD. Pre-sales prototypes get a quick check."	A new plan from notes in minutes (D); cost per run; the 3 apps already onboarded (Demo Shop, EcoForecast, ForecastIQ Legacy); next-step pilot	The "60%" is an estimate: bring the measured basis
Innovation and Creativity (20%)	"AI used in a layered way: a fast cheap model for each click, a strong model only when unsure; goal-driven journeys; replay at zero model cost; calculations verified by code."	"How it decided" tab (fast vs strong calls), *Repeat exactly*, the expected-outcome numbers	Do not over-claim "autonomous"; say "plain English in, proof out"
Novelty and Problem Relevance (15%)	"Pharma commercial analytics lives and dies on numbers being right. Generic UI tests check that a page loads; we check that the forecast is right."	The 422/"144 invented rows" bug; 6,786 to 6,889 expectation	Be ready for question 23
Presentation, Demo and Q&A (15%)	Clear arc: problem, solution, live demo, value, next step. Tight timing, one backup.	The run of show above; the backup video; this Q&A list	Over-running the demo: cut G and shorten B

The organisers' "winning formula" is: *Real Problem + Practical Solution + Working POC + Chryselys Business Value + Reusable Potential.* Tie your closing line to it: "a real problem, a working POC on our own product, and a reusable accelerator ready for a pilot."

Their "Test": *can someone outside your team understand the problem and see the solution working in a few minutes?* Keep jargon out. Say "test plan", "journey", "bug", not "LangGraph" or "tape".

7. Suggested tweaks to the two slides

- Slide 1: "~60% (est.)" is fine; add the basis in speaker notes. "0 code" is better said as "no scripting".
- Slide 2: the dashed "Live Demo" box is good; add one line, "Also: test just one flow, or a pre-sales prototype".
- Both are already within the "1-2 intro slides" rule.